

Complete Guide to `grep`, `sed`, `awk`, and `find` in Bash Scripting

Table of Contents

1. [Introduction](#)
 2. [Regular Expressions](#)
 3. [grep Command](#)
 4. [sed Command](#)
 5. [awk Command](#)
 6. [find Command](#)
 7. [Combining grep, sed, awk, and find](#)
 8. [Practical Bash Scripts](#)
 9. [Common Mistakes and Safety Guidelines](#)
 10. [Quick Reference Cheat Sheet](#)
-

1. Introduction

Four of the most important text-processing and file-searching tools in Bash are:

Command	Main purpose
<code>grep</code>	Searches text for matching patterns
<code>sed</code>	Edits and transforms text streams
<code>awk</code>	Processes structured text using fields, records, conditions, and calculations
<code>find</code>	Searches the filesystem for files and directories

Although these commands sometimes overlap, each one is designed for a different type of task.

Basic comparison

Suppose a file named `employees.txt` contains:

```
101 Alice Engineering 75000
102 Bob Sales 62000
103 Carol Engineering 82000
104 David Marketing 58000
105 Eva Sales 68000
```

You could use `grep` to find employees in Engineering:

```
grep "Engineering" employees.txt
```

You could use `sed` to replace `Engineering` with `Development`:

```
sed 's/Engineering/Development/g' employees.txt
```

You could use `awk` to print names and salaries:

```
awk '{ print $2, $4 }' employees.txt
```

You could use `find` to locate the file:

```
find . -name "employees.txt"
```

2. Regular Expressions

Regular expressions, usually called regex, are patterns used for matching text.

They are heavily used by:

- `grep`
- `sed`
- `awk`
- `find -regex`

2.1 Literal text

A normal word matches itself.

```
grep "error" application.log
```

This searches for the literal text `error`.

2.2 The dot character

A dot matches any single character.

```
c.t
```

This can match:

```
cat  
cot  
cut  
c9t
```

It does not match:

```
ct  
coat
```

2.3 Beginning of a line

The caret `^` represents the beginning of a line.

```
grep '^ERROR' application.log
```

This matches lines beginning with `ERROR`.

Example:

```
ERROR Database unavailable  
ERROR Login failed
```

It does not match:

```
An ERROR occurred
```

2.4 End of a line

The dollar sign `$` represents the end of a line.

```
grep 'completed$' application.log
```

This matches lines ending with `completed`.

2.5 Character classes

Square brackets match one character from a group.

```
[aeiou]
```

Matches one lowercase vowel.

```
[0-9]
```

Matches one digit.

```
[a-z]
```

Matches one lowercase letter.

```
[A-Z]
```

Matches one uppercase letter.

```
[A-Za-z]
```

Matches one English alphabetic character.

Example:

```
grep '[0-9]' data.txt
```

This prints lines containing at least one digit.

2.6 Negated character classes

A caret inside square brackets negates the class.

```
[^0-9]
```

Matches one character that is not a digit.

```
[^aeiou]
```

Matches one character that is not a lowercase vowel.

2.7 Zero or more repetitions

The asterisk `*` means zero or more repetitions of the previous pattern.

```
ab*
```

Matches:

```
a  
ab  
abb  
abbb
```

A common regex is:

```
.*
```

This means any character repeated zero or more times.

It can match any amount of text, including an empty string.

2.8 One or more repetitions

In extended regular expressions, `+` means one or more repetitions.

```
[0-9]+
```

Matches one or more digits.

Use `grep -E` for extended regular expressions:

```
grep -E '[0-9]+' data.txt
```

In standard `sed`, you may need:

```
sed -E 's/[0-9]+/NUMBER/g' file.txt
```

On some older GNU systems, `sed -r` is also used, but `-E` is more portable across modern systems.

2.9 Zero or one occurrence

In extended regular expressions, `?` means zero or one occurrence.

```
colou?r
```

Matches both:

```
color  
colour
```

Example:

```
grep -E 'colou?r' file.txt
```

2.10 Alternation

The pipe symbol `|` means OR in extended regular expressions.

```
grep -E 'error|warning|critical' application.log
```

This matches lines containing any of:

- error
- warning
- critical

2.11 Grouping

Parentheses group patterns in extended regular expressions.

```
(error|warning):[0-9]+
```

Example:

```
grep -E '(ERROR|WARNING):[0-9]+' application.log
```

2.12 Repetition counts

Extended regular expressions support exact repetition counts.

```
[0-9]{4}
```

Matches exactly four digits.

```
[0-9]{2,4}
```

Matches between two and four digits.

```
[0-9]{3,}
```

Matches three or more digits.

Example:

```
grep -E '^'[0-9]{4}$' years.txt
```

This matches lines containing exactly four digits.

2.13 POSIX character classes

POSIX classes are often more portable than ranges such as `[A-Z]`.

Class	Meaning
<code>[:digit:]</code>	Digit
<code>[:alpha:]</code>	Alphabetic character
<code>[:alnum:]</code>	Letter or digit
<code>[:space:]</code>	Whitespace
<code>[:upper:]</code>	Uppercase letter
<code>[:lower:]</code>	Lowercase letter
<code>[:punct:]</code>	Punctuation
<code>[:blank:]</code>	Space or tab

Example:

```
grep '[:digit:]' file.txt
```

2.14 Escaping special characters

To search for a literal dot, escape it:

```
grep '\.' file.txt
```

Without escaping, `.` means any character.

To search for text such as:

```
192.168.1.10
```

Use:

```
grep '192\.168\.1\.10' file.txt
```

3. `grep` Command

3.1 What is `grep`?

`grep` searches input text and prints lines matching a pattern.

The name is historically associated with:

```
Global Regular Expression Print
```

General syntax:

```
grep [options] pattern [file...]
```

Example:

```
grep "ERROR" application.log
```


Here:

- `grep` is the command.
 - `"ERROR"` is the search pattern.
 - `application.log` is the input file.
-

3.2 Basic search

File:

```
Linux is powerful.  
Bash is a shell.  
Linux supports many shells.  
Python is a language.
```

Command:

```
grep "Linux" file.txt
```

Output:

```
Linux is powerful.  
Linux supports many shells.
```

`grep` prints complete matching lines, not only the matched word.

3.3 Case-insensitive search

Use `-i`:

```
grep -i "linux" file.txt
```

This matches:

```
linux  
Linux  
LINUX  
LiNuX
```

3.4 Display line numbers

Use `-n`:

```
grep -n "Linux" file.txt
```

Output:

```
1:Linux is powerful.  
3:Linux supports many shells.
```

The number before the colon is the line number.

3.5 Invert the match

Use `-v` to print lines that do not match.

```
grep -v "Linux" file.txt
```

Output:

```
Bash is a shell.  
Python is a language.
```

A common use is removing blank lines:

```
grep -v '^$' file.txt
```

A better version that also removes lines containing only whitespace is:

```
grep -v '^[[:space:]]*$' file.txt
```

3.6 Count matching lines

Use `-c`:

```
grep -c "ERROR" application.log
```

Possible output:

25

This means 25 lines matched.

It does not necessarily mean the word appeared exactly 25 times. One line could contain the word multiple times.

To count every occurrence using GNU `grep`:

```
grep -o "ERROR" application.log | wc -l
```

Explanation:

1. `grep -o "ERROR"` prints every matching occurrence separately.
2. `wc -l` counts the output lines.

3.7 Print only matching text

Use `-o`:

```
echo "IDs: 123, 456, 789" | grep -oE '[0-9]+'
```

Output:

```
123
456
789
```

Without `-o`, `grep` would print the complete input line.

3.8 Match complete words

Use `-w`:

```
grep -w "cat" file.txt
```

This matches:

```
The cat is sleeping.
```

It does not normally match:

```
category
concatenate
bobcat
```

3.9 Match complete lines

Use `-x`:

```
grep -x "ERROR" file.txt
```

This matches only lines whose entire content is exactly:

```
ERROR
```

It does not match:

```
ERROR occurred
```

3.10 Extended regular expressions

Use `-E`:

```
grep -E 'ERROR|WARNING|CRITICAL' application.log
```

Without `-E`, the `|` character may not behave as alternation unless escaped.

Older command:

```
egrep 'ERROR|WARNING' application.log
```

Modern recommendation:

```
grep -E 'ERROR|WARNING' application.log
```

3.11 Fixed-string search

Use `-F` when the pattern should be treated literally.

```
grep -F '192.168.1.10' access.log
```

With `-F`, characters such as `.` and `*` lose their regex meanings.

This is useful for:

- IP addresses
- URLs
- Source-code fragments
- Strings containing brackets
- Strings containing regex metacharacters

Older command:

```
fgrep 'literal.text' file.txt
```

Modern recommendation:

```
grep -F 'literal.text' file.txt
```

3.12 Search recursively

Use `-r`:

```
grep -r "TODO" .
```

This searches all files under the current directory.

To include line numbers:

```
grep -rn "TODO" .
```

To ignore case:

```
grep -rni "todo" .
```

A commonly used combination is:

```
grep -Rni "search_text" directory/
```

`-R` follows symbolic links, while `-r` generally does not follow all symbolic links.

3.13 Include only certain file types

```
grep -rn --include='*.py' "TODO" .
```

This searches only Python files.

Multiple include patterns can be used:

```
grep -rn
  --include='*.py'
  --include='*.sh'
  "TODO" .
```

3.14 Exclude file types

```
grep -rn --exclude='*.log' "password" .
```

This excludes files ending in `.log`.

Exclude multiple patterns:

```
grep -rn
  --exclude='*.log'
  --exclude='*.tmp'
  "password" .
```

3.15 Exclude directories

```
grep -rn
  --exclude-dir='.git'
  --exclude-dir='node_modules'
  "TODO" .
```

This avoids searching Git metadata and Node.js dependencies.

3.16 Show matching context

Lines before a match

```
grep -B 2 "ERROR" application.log
```

`-B 2` prints two lines before every matching line.

Lines after a match

```
grep -A 3 "ERROR" application.log
```

`-A 3` prints three lines after every matching line.

Lines before and after

```
grep -C 2 "ERROR" application.log
```

`-C 2` prints two lines before and two lines after.

3.17 Search multiple files

```
grep "ERROR" app.log system.log database.log
```

Output may look like:

```
app.log:ERROR Login failed
database.log:ERROR Connection refused
```

When multiple files are searched, `grep` normally prints each filename.

3.18 Print filenames containing matches

Use `-l`:

```
grep -rl "TODO" .
```

This prints only filenames that contain at least one match.

Example:

```
./script.sh  
./main.py  
./README.md
```

3.19 Print filenames without matches

Use `-L`:

```
grep -rL "copyright" .
```

This prints files that do not contain the pattern.

3.20 Suppress filenames

Use `-h`:

```
grep -h "ERROR" *.log
```

Even when searching multiple files, filenames will not be displayed.

3.21 Force filenames to be shown

Use `-H`:

```
grep -H "ERROR" application.log
```

This prints the filename even when only one file is searched.

3.22 Limit matches

GNU `grep` supports `-m`:

```
grep -m 3 "ERROR" application.log
```

This stops after three matching lines per input file.

3.23 Read patterns from a file

Suppose `patterns.txt` contains:

```
ERROR
WARNING
CRITICAL
```

Use:

```
grep -f patterns.txt application.log
```

This searches for all patterns listed in `patterns.txt`.

For literal patterns:

```
grep -Ff patterns.txt application.log
```

3.24 Multiple patterns with `-e`

```
grep -e "ERROR" -e "WARNING" application.log
```

This matches either `ERROR` or `WARNING`.

This is useful when a pattern begins with `-`.

```
grep -e "-debug" file.txt
```

Another solution is:

```
grep -- "-debug" file.txt
```

The `--` indicates the end of command options.

3.25 Search standard input

`grep` can process output produced by another command.

```
ps aux | grep "nginx"
```

Explanation:

1. `ps aux` prints running processes.
2. `|` sends the output to `grep`.
3. `grep "nginx"` prints lines containing `nginx`.

A common problem is that this may also match the `grep` command itself.

One workaround:

```
ps aux | grep '[n]ginx'
```

A better process-search command is often:

```
pgrep -af nginx
```

3.26 Quiet mode

Use `-q` when you only care whether a match exists.

```
if grep -q "ERROR" application.log; then
    echo "Errors were found."
else
    echo "No errors were found."
fi
```

`grep -q` prints nothing.

Instead, the script checks the command's exit status.

3.27 `grep` exit status

`grep` uses these important exit codes:

Exit status	Meaning
0	At least one match was found
1	No match was found
2 or greater	An error occurred

Example:

```
grep -q "root" /etc/passwd

status=$?

echo "$status"
```

3.28 Searching for email-like text

```
grep -Eo '[:alnum:]._%+~]+@[[:alnum:].-]+\.[[:alpha:]]{2,}' file.txt
```

Explanation:

- `[:alnum:]._%+~]+` matches the username-like portion.
- `@` matches a literal at sign.
- `[:alnum:].-]+` matches the domain.
- `\.` matches a literal dot.
- `[[:alpha:]]{2,}` matches a domain suffix of at least two letters.

This is a practical pattern, not a perfect implementation of every valid email-address rule.

3.29 Searching for IPv4-like addresses

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log
```

This finds patterns shaped like IPv4 addresses.

However, it will also match invalid values such as:

```
999.999.999.999
```

Validating the numerical range requires additional logic.

3.30 Useful `grep` examples

Find comment lines:

```
grep '^[:space:]*#' script.sh
```

Find non-comment, non-empty lines:

```
grep -Ev '^[[:space:]]*(#|$)' configuration.conf
```

Find lines containing only numbers:

```
grep -E '^[0-9]+$' file.txt
```

Find lines beginning with a number:

```
grep -E '^[0-9]' file.txt
```

Find lines ending with `.com` :

```
grep '\.com$' domains.txt
```

Search case-insensitively with line numbers:

```
grep -in "failed" application.log
```

Search recursively in shell scripts:

```
grep -rn --include='*.sh' "rm -rf" .
```

4. `sed` Command

4.1 What is `sed`?

`sed` means stream editor.

It reads text line by line, applies editing instructions, and prints the result.

General syntax:

```
sed [options] 'command' file
```

Example:

```
sed 's/Linux/Unix/' file.txt
```

This replaces the first occurrence of `Linux` with `Unix` on every line.

By default, `sed` does not modify the original file. It prints the transformed result to standard output.

4.2 Substitution syntax

The most common `sed` command is `s`, meaning substitute.

```
sed 's/old/new/' file.txt
```

Structure:

```
s / search_pattern / replacement /
```

Example input:

```
Linux is powerful.  
I use Linux every day.
```

Command:

```
sed 's/Linux/Unix/' file.txt
```

Output:

```
Unix is powerful.  
I use Unix every day.
```

4.3 Replace all occurrences on each line

By default, `sed` replaces only the first occurrence on each line.

Input:

```
Linux Linux Linux
```

Command:

```
sed 's/Linux/Unix/' file.txt
```

Output:

```
Unix Linux Linux
```

Use the `g` flag to replace all occurrences:

```
sed 's/Linux/Unix/g' file.txt
```

Output:

```
Unix Unix Unix
```

4.4 Replace only a specific occurrence

Replace the second occurrence on every line:

```
sed 's/Linux/Unix/2' file.txt
```

Input:

```
Linux Linux Linux
```

Output:

```
Linux Unix Linux
```

Replace the third occurrence:

```
sed 's/Linux/Unix/3' file.txt
```

4.5 Case-insensitive replacement

GNU `sed` supports the `I` flag:

```
sed 's/linux/Unix/gI' file.txt
```

This can replace:

```
linux  
Linux  
LINUX
```

Portability note: case-insensitive flags can vary between `sed` implementations.

4.6 Print only changed lines

Use `-n` to disable automatic printing and `p` to print successful substitutions.

```
sed -n 's/Linux/Unix/p' file.txt
```

Only lines where the replacement occurred will be printed.

4.7 Alternative delimiters

You do not have to use `/` as the delimiter.

For paths, `/` can make the command difficult to read.

Instead of:

```
sed 's/\\home\\user\\old\\/\\home\\user\\new/g' file.txt
```

Use:

```
sed 's|/home/user/old|/home/user/new|g' file.txt
```

Other delimiters can also be used:

```
sed 's#/old/path#/new/path#g' file.txt
```

4.8 Print specific lines

Print line 5:

```
sed -n '5p' file.txt
```

Print lines 3 through 7:

```
sed -n '3,7p' file.txt
```

Print from line 10 to the end:

```
sed -n '10,$p' file.txt
```

Here, `$` means the last line.

4.9 Delete lines

Delete line 3:

```
sed '3d' file.txt
```

Delete lines 3 through 7:

```
sed '3,7d' file.txt
```

Delete the last line:

```
sed '$d' file.txt
```

Delete blank lines:

```
sed '/^$/d' file.txt
```

Delete whitespace-only lines:

```
sed '/^[[:space:]]*$/d' file.txt
```

Delete comment lines:


```
sed '/^[[:space:]]*#/d' file.txt
```

Delete lines containing `DEBUG`:

```
sed '/DEBUG/d' application.log
```

Delete lines not containing `ERROR`:

```
sed '/ERROR/!d' application.log
```

The `!` negates the address condition.

4.10 Addressing lines by pattern

Print lines containing `ERROR`:

```
sed -n '/ERROR/p' application.log
```

Delete lines containing `DEBUG`:

```
sed '/DEBUG/d' application.log
```

Replace text only on matching lines:

```
sed '/ERROR/s/failed/FAILED/g' application.log
```

Explanation:

- `/ERROR/` selects lines containing `ERROR`.
 - `s/failed/FAILED/g` performs the replacement only on those lines.
-

4.11 Address ranges using patterns

Print from `START` to `END`:

```
sed -n '/START/,/END/p' file.txt
```

Delete a section:

```
sed '/BEGIN TEMP/,/END TEMP/d' file.txt
```

The range begins when a line matches the first pattern and ends when a line matches the second pattern.

4.12 Replace text only on a line range

```
sed '10,20s/error/ERROR/g' file.txt
```

This performs replacement only from line 10 through line 20.

4.13 Insert text before a line

Insert before line 3:

```
sed '3i  
New line  
' file.txt
```

GNU `sed` also commonly accepts:

```
sed '3i New line' file.txt
```

Insert before lines containing `ERROR`:

```
sed '/ERROR/i  
----- Error detected -----  
' application.log
```

4.14 Append text after a line

Append after line 3:

```
sed '3a  
New line  
' file.txt
```

Append after lines containing `ERROR`:

```
sed '/ERROR/a
Please investigate this error.
' application.log
```

4.15 Change an entire line

Replace line 4 completely:

```
sed '4c
This is the new fourth line.
' file.txt
```

Replace every line containing `deprecated`:

```
sed '/deprecated/c
This feature has been removed.
' file.txt
```

4.16 Multiple `sed` commands

Use multiple `-e` options:

```
sed
-e 's/Linux/Unix/g'
-e 's/Bash/Shell/g'
file.txt
```

Or separate commands with semicolons:

```
sed 's/Linux/Unix/g; s/Bash/Shell/g' file.txt
```

For longer scripts, use a multiline block:

```
sed '
s/Linux/Unix/g
s/Bash/Shell/g
/^[[:space:]]*$/d
' file.txt
```

4.17 Edit a file in place

GNU `sed`:

```
sed -i 's/old/new/g' file.txt
```

This modifies `file.txt`.

Create a backup:

```
sed -i.bak 's/old/new/g' file.txt
```

This creates:

```
file.txt  
file.txt.bak
```

The backup contains the original content.

macOS/BSD `sed`

On macOS, in-place editing often requires an explicit backup suffix:

```
sed -i '' 's/old/new/g' file.txt
```

With a backup:

```
sed -i '.bak' 's/old/new/g' file.txt
```

For portable scripts, be careful with `sed -i`.

4.18 Capture groups and backreferences

Suppose the input contains:

```
John Smith  
Alice Brown
```

Swap first and last names:

```
sed -E 's/^[^ ]+ ([^ ]+)$/ \2 \1/' names.txt
```

Output:

```
Smith John  
Brown Alice
```

Explanation:

```
^
```

Beginning of line.

```
([ ^ ]+)
```

Capture one or more non-space characters as group 1.

```
([ ^ ]+)
```

A space followed by another non-space sequence as group 2.

```
$
```

End of line.

Replacement:

```
\2 \1
```

Print group 2 first, followed by group 1.

4.19 The `&` replacement symbol

In a replacement, `&` represents the complete matched text.

Input:

```
Salary: 50000  
Salary: 70000
```

Command:

```
sed -E 's/[0-9]+/[$&]/g' file.txt
```

Output:

```
Salary: [$50000]  
Salary: [$70000]
```

A clearer example:

```
sed -E 's/[0-9]+/[/g' file.txt
```

Output:

```
Salary: [50000]  
Salary: [70000]
```

4.20 Add a prefix to every line

```
sed 's/^/PREFIX: /' file.txt
```

Input:

```
apple  
banana
```

Output:

```
PREFIX: apple  
PREFIX: banana
```

4.21 Add a suffix to every line

```
sed 's/$/ :END/' file.txt
```

Output:

```
apple :END  
banana :END
```

4.22 Remove leading whitespace

```
sed 's/^[[:space:]]*//' file.txt
```

Explanation:

- `^` means beginning of line.
- `[[:space:]]*` matches zero or more whitespace characters.
- The replacement is empty.

4.23 Remove trailing whitespace

```
sed 's/[[:space:]]*$//' file.txt
```

4.24 Trim both leading and trailing whitespace

```
sed  
-e 's/^[[:space:]]*//'  
-e 's/[[:space:]]*$//'  
file.txt
```

4.25 Replace repeated spaces

Using extended regular expressions:

```
sed -E 's/[[:space:]]+/ /g' file.txt
```

This converts one or more whitespace characters into one space.

Be careful: it also affects tabs and possibly leading indentation.

4.26 Remove HTML-like tags

```
sed -E 's/<[^>]*>//g' file.html
```

Input:

```
<p>Hello <strong>world</strong></p>
```

Output:

```
Hello world
```

This is suitable only for simple text transformations. Regular expressions are not a complete HTML parser.

4.27 Replace the first line

```
sed '1s/.*/New first line/' file.txt
```

4.28 Number lines with `sed`

```
sed = file.txt
```

This produces line numbers and text on separate lines:

```
1
First line
2
Second line
```

To place them on the same line:

```
sed = file.txt | sed 'N;s/\n/ /'
```

However, `nl` or `awk` is normally better:

```
nl -ba file.txt
```


or:

```
awk '{ print NR, $0 }' file.txt
```

4.29 Read commands from a file

Create `commands.sed`:

```
s/Linux/Unix/g  
s/Bash/Shell/g  
/^[[:space:]]*$/d
```

Run:

```
sed -f commands.sed file.txt
```

This is useful for complicated transformations.

4.30 Pattern space

`sed` reads one line into an internal area called the pattern space.

A simplified processing cycle is:

1. Read one input line into the pattern space.
2. Execute the `sed` commands.
3. Print the pattern space unless `-n` is used.
4. Clear the pattern space.
5. Read the next line.

Most normal `sed` commands operate on the pattern space.

4.31 Hold space

`sed` also has another internal storage area called the hold space.

Important commands:

Command	Meaning
<code>h</code>	Copy pattern space to hold space

Command	Meaning
<code>H</code>	Append pattern space to hold space
<code>g</code>	Copy hold space to pattern space
<code>G</code>	Append hold space to pattern space
<code>x</code>	Exchange pattern and hold spaces

Example: reverse pairs of lines:

```
sed -n 'h;n;G;p' file.txt
```

Suppose the file contains:

```
one
two
three
four
```

Output:

```
two
one
four
three
```

Explanation:

1. `h` saves the first line in hold space.
2. `n` reads the next line into pattern space.
3. `G` appends the saved line.
4. `p` prints both lines.

This is an advanced `sed` technique.

4.32 Multiline processing with `N`

The `N` command appends the next input line to the pattern space.

Suppose the file contains:

```
Hello
World
```

Command:

```
sed 'N;s/Hello\nWorld/Hello World/' file.txt
```

Output:

```
Hello World
```

The pattern space temporarily contains:

```
Hello\nWorld
```

4.33 The `n` command

The lowercase `n` prints the current pattern space, then reads the next line.

Example:

```
sed 'n;d' file.txt
```

This can be used to print alternating lines depending on command structure.

A clearer way to print even-numbered lines is:

```
sed -n '2~2p' file.txt
```

The `~` address is a GNU extension.

Portable alternative:

```
awk 'NR % 2 == 0' file.txt
```

4.34 Branching

`sed` supports labels and branches.

Important commands:

Command	Meaning
<code>:label</code>	Define a label
<code>b label</code>	Branch unconditionally
<code>t label</code>	Branch if a substitution succeeded

Example: repeatedly replace double spaces until none remain:

```
sed ':again; s/  / /g; t again' file.txt
```

Explanation:

1. `:again` defines a label.
2. `s/ / /g` replaces double spaces with one space.
3. `t again` jumps back if a replacement occurred.
4. The loop continues until no double spaces remain.

A modern extended-regex solution is simpler:

```
sed -E 's/ +/ /g' file.txt
```

4.35 Useful `sed` examples

Remove Windows carriage returns:

```
sed 's/\r$//' file.txt
```

Remove everything after `#`:

```
sed 's/#.*//' file.txt
```

Remove comments and blank lines:

```
sed
-e 's/#.*//'
-e '/^[:space:]*$/d'
configuration.conf
```

Print lines between two markers:

```
sed -n '/BEGIN/,/END/p' file.txt
```

Print every line except the first:

```
sed '1d' file.txt
```

Print every line except the last:

```
sed '$d' file.txt
```

Replace only the first occurrence in the entire file:

```
sed '0,/old/s//new/' file.txt
```

This is a GNU `sed` extension.

5. `awk` Command

5.1 What is `awk` ?

`awk` is a text-processing language.

It is especially useful for:

- Column-based data
- Reports
- Calculations
- Filtering
- Aggregation
- Reformatting
- Log processing
- Associative arrays
- Small data-processing programs

General syntax:

```
awk 'pattern { action }' file
```

Example:

```
awk '{ print $1 }' file.txt
```

This prints the first field of every line.

5.2 Records and fields

By default:

- Each input line is one record.
- Each whitespace-separated value is one field.

Suppose `employees.txt` contains:

```
101 Alice Engineering 75000
102 Bob Sales 62000
103 Carol Engineering 82000
```

For the first line:

Variable	Value
<code>\$0</code>	101 Alice Engineering 75000
<code>\$1</code>	101
<code>\$2</code>	Alice
<code>\$3</code>	Engineering
<code>\$4</code>	75000
<code>NF</code>	4
<code>NR</code>	1

5.3 Print complete lines

```
awk '{ print $0 }' employees.txt
```

Because `$0` represents the full current record, this prints the complete file.

The following also prints every line:

```
awk '{ print }' employees.txt
```

5.4 Print specific fields

Print employee names:

```
awk '{ print $2 }' employees.txt
```

Output:

```
Alice  
Bob  
Carol
```

Print names and departments:

```
awk '{ print $2, $3 }' employees.txt
```

Output:

```
Alice Engineering  
Bob Sales  
Carol Engineering
```

A comma in `print` inserts the output field separator.

5.5 Concatenate fields

```
awk '{ print $2 "-" $3 }' employees.txt
```

Output:

```
Alice-Engineering  
Bob-Sales  
Carol-Engineering
```

There is no comma between `$2`, `"-"`, and `$3`, so they are concatenated.

5.6 Add labels

```
awk '{ print "Name:", $2, "Salary:", $4 }' employees.txt
```

Output:

```
Name: Alice Salary: 75000
Name: Bob Salary: 62000
Name: Carol Salary: 82000
```

5.7 Field separator with `-F`

Suppose `users.csv` contains:

```
101,Alice,Engineering,75000
102,Bob,Sales,62000
103,Carol,Engineering,82000
```

Use a comma as the field separator:

```
awk -F ',' '{ print $2, $4 }' users.csv
```

Output:

```
Alice 75000
Bob 62000
Carol 82000
```

A shorter form is:

```
awk -F, '{ print $2, $4 }' users.csv
```

5.8 Set `FS` inside `awk`

```
awk 'BEGIN { FS = "," } { print $2, $4 }' users.csv
```

`FS` means input field separator.

The `BEGIN` block runs before input processing begins.

5.9 Output field separator

`OFS` means output field separator.

```
awk 'BEGIN { FS = ","; OFS = " | " } { print $2, $3, $4 }' users.csv
```

Output:

```
Alice | Engineering | 75000
Bob | Sales | 62000
Carol | Engineering | 82000
```

5.10 Record separator

`RS` means input record separator.

Normally, a record is one line because `RS` defaults to a newline.

Suppose `records.txt` contains:

```
Alice:Engineering:75000;Bob:Sales:62000;Carol:Engineering:82000
```

Process semicolon-separated records:

```
awk 'BEGIN { RS = ";"; FS = ":" } { print $1, $3 }' records.txt
```

Output:

```
Alice 75000
Bob 62000
Carol 82000
```

5.11 Output record separator

`ORS` means output record separator.

```
awk 'BEGIN { ORS = " | " } { print $1 }' employees.txt
```

Output:

```
101 | 102 | 103 |
```

5.12 NR and FNR

NR is the total record number across all input files.

```
awk '{ print NR, $0 }' file.txt
```

Output:

```
1 First line
2 Second line
3 Third line
```

FNR is the record number within the current file.

Command:

```
awk '{ print FILENAME, NR, FNR, $0 }' file1.txt file2.txt
```

If each file contains three lines:

```
file1.txt 1 1 ...
file1.txt 2 2 ...
file1.txt 3 3 ...
file2.txt 4 1 ...
file2.txt 5 2 ...
file2.txt 6 3 ...
```

NR continues increasing, while **FNR** resets for each file.

5.13 NF

NF is the number of fields in the current record.

```
awk '{ print NF, $0 }' employees.txt
```

Print the last field:

```
awk '{ print $NF }' employees.txt
```

Why does `$NF` work?

- `NF` contains the number of fields.
- If `NF` is `4`, `$NF` means `$4`.

Print the second-last field:

```
awk '{ print $(NF - 1) }' employees.txt
```

5.14 Pattern matching

Print lines containing `Engineering`:

```
awk '/Engineering/' employees.txt
```

This is equivalent to:

```
awk '/Engineering/ { print $0 }' employees.txt
```

Print only names from matching lines:

```
awk '/Engineering/ { print $2 }' employees.txt
```

5.15 Match a specific field

```
awk '$3 == "Engineering" { print $2, $4 }' employees.txt
```

Output:

```
Alice 75000  
Carol 82000
```

This checks whether field 3 is exactly `Engineering`.

5.16 Regular-expression operators

`~` means matches regex.

```
awk '$2 ~ /^A/ { print $2 }' employees.txt
```

This prints names beginning with `A`.

`!~` means does not match regex.

```
awk '$3 !~ /Sales/ { print $2, $3 }' employees.txt
```

5.17 Numeric conditions

Print employees earning more than 70,000:

```
awk '$4 > 70000 { print $2, $4 }' employees.txt
```

Print salaries between 60,000 and 80,000:

```
awk '$4 >= 60000 && $4 <= 80000 { print $2, $4 }' employees.txt
```

5.18 Logical operators

Operator	Meaning
<code>&&</code>	Logical AND
<code> </code>	Logical OR
<code>!</code>	Logical NOT

Example:

```
awk '$3 == "Engineering" && $4 > 75000 { print $2 }' employees.txt
```

Example using OR:

```
awk '$3 == "Sales" || $3 == "Marketing" { print $2, $3 }' employees.txt
```

5.19 BEGIN block

A `BEGIN` block runs before any input records are read.

```
awk 'BEGIN { print "Employee Report" } { print $2, $4 }' employees.txt
```

Output:

```
Employee Report
Alice 75000
Bob 62000
Carol 82000
```

`BEGIN` is commonly used to:

- Print headers
- Set separators
- Initialize variables
- Process command-line arguments
- Perform calculations without an input file

Example:

```
awk 'BEGIN { print 10 + 20 }'
```

Output:

```
30
```

5.20 END block

An `END` block runs after all input records have been processed.

```
awk '
BEGIN {
    print "Employee Report"
}
{
    print $2, $4
}
END {
    print "End of Report"
}
```

```
}  
' employees.txt
```

5.21 Variables

```
awk '  
{  
    salary = $4  
    bonus = salary * 0.10  
    print $2, salary, bonus  
}  
' employees.txt
```

Output:

```
Alice 75000 7500  
Bob 62000 6200  
Carol 82000 8200
```

`awk` variables do not need to be declared before use.

5.22 Sum values

Calculate total salary:

```
awk '{ total += $4 } END { print "Total salary:", total }' employees.txt
```

Explanation:

1. `total` initially behaves like zero.
2. For each record, `$4` is added to `total`.
3. The `END` block prints the final total.

5.23 Calculate an average

```
awk '  
{  
    total += $4  
    count++  
}  
END {
```

```
    if (count > 0) {
        print "Average salary:", total / count
    }
}
' employees.txt
```

The `count > 0` check prevents division by zero for an empty file.

You could also use `NR` when every input line is a valid record:

```
awk '{ total += $4 } END { if (NR > 0) print total / NR }' employees.txt
```

5.24 Minimum and maximum

Find the maximum salary:

```
awk '
NR == 1 {
    max = $4
    employee = $2
}
$4 > max {
    max = $4
    employee = $2
}
END {
    print employee, max
}
' employees.txt
```

Explanation:

- On the first line, initialize `max`.
- For every following line, compare the current salary with `max`.
- If the current salary is greater, update `max` and `employee`.

A compact version:

```
awk '
NR == 1 || $4 > max {
    max = $4
    employee = $2
}
END {
    print employee, max
}
```

```
}  
' employees.txt
```

Find the minimum salary:

```
awk '  
NR == 1 || $4 < min {  
    min = $4  
    employee = $2  
}  
END {  
    print employee, min  
}  
' employees.txt
```

5.25 if, else if, and else

```
awk '  
{  
    if ($4 >= 80000) {  
        category = "High"  
    } else if ($4 >= 60000) {  
        category = "Medium"  
    } else {  
        category = "Low"  
    }  
  
    print $2, $4, category  
}  
' employees.txt
```

5.26 Ternary operator

Syntax:

```
condition ? value_if_true : value_if_false
```

Example:

```
awk '{  
    status = ($4 >= 70000) ? "Above target" : "Below target"
```



```
    print $2, status
}' employees.txt
```

5.27 for loop

```
awk '
{
    for (i = 1; i <= NF; i++) {
        print "Field", i, "=", $i
    }
}' employees.txt
```

For every record, this prints every field.

5.28 while loop

```
awk '
{
    i = 1

    while (i <= NF) {
        print $i
        i++
    }
}' employees.txt
```

5.29 do-while loop

```
awk '
BEGIN {
    i = 1

    do {
        print i
        i++
    } while (i <= 5)
}'
```

Output:

```
1  
2  
3  
4  
5
```

5.30 Arrays

`awk` arrays are associative arrays.

```
awk '
BEGIN {
    fruits["red"] = "apple"
    fruits["yellow"] = "banana"
    fruits["green"] = "lime"

    print fruits["yellow"]
}
```

Output:

```
banana
```

The index does not need to be a number.

5.31 Count records by category

Count employees per department:

```
awk '
{
    department_count[$3]++
}
END {
    for (department in department_count) {
        print department, department_count[department]
    }
}
' employees.txt
```

Possible output:

```
Engineering 2  
Sales 2  
Marketing 1
```

The order may vary because normal `awk` associative arrays are not necessarily iterated in sorted order.

5.32 Sum values by category

Calculate salary totals by department:

```
awk '
{
    salary_total[$3] += $4
}
END {
    for (department in salary_total) {
        print department, salary_total[department]
    }
}
' employees.txt
```

5.33 Average by category

```
awk '
{
    total[$3] += $4
    count[$3]++
}
END {
    for (department in total) {
        average = total[department] / count[department]
        print department, average
    }
}
' employees.txt
```

5.34 Delete an array element

```
delete array[index]
```

Example:

```
awk '
BEGIN {
    data["one"] = 1
    data["two"] = 2

    delete data["one"]

    print data["two"]
}
'
```

GNU `awk` also supports deleting the complete array:

```
delete data
```

This is not supported by every older `awk` implementation.

5.35 String length

```
awk '{ print $2, length($2) }' employees.txt
```

Output:

```
Alice 5
Bob 3
Carol 5
```

`length()` without an argument returns the length of `$0`.

```
awk '{ print length() }' file.txt
```

5.36 Extract part of a string

Use `substr()`:

```
substr(string, start, length)
```

Example:

```
awk '{ print substr($2, 1, 3) }' employees.txt
```

For `Alice`, this returns:

```
Ali
```

`awk` string indexes begin at 1.

5.37 Find text inside a string

Use `index()`:

```
awk '{
    position = index($0, "Engineering")
    print position
}' employees.txt
```

`index()` returns:

- The character position if found
 - `0` if not found
-

5.38 Convert case

Standard POSIX `awk` supports:

```
toupper(string)
tolower(string)
```

Examples:

```
awk '{ print toupper($2) }' employees.txt
```

```
awk '{ print tolower($3) }' employees.txt
```

5.39 Replace text with `sub()`

`sub()` replaces the first match.

```
awk '{
    sub(/Engineering/, "Development")
    print
}' employees.txt
```

By default, `sub()` modifies `$0`.

To modify a specific field:

```
awk '{
    sub(/Engineering/, "Development", $3)
    print
}' employees.txt
```

5.40 Replace all matches with `gsub()`

```
awk '{
    gsub(/a/, "A")
    print
}' file.txt
```

To replace inside a particular field:

```
awk '{
    gsub(/Sales/, "Business", $3)
    print
}' employees.txt
```

5.41 Split a string

```
split(string, array, separator)
```

Example:

```
awk '
BEGIN {
    text = "apple,banana,mango"

    count = split(text, fruits, ",")

    for (i = 1; i <= count; i++) {
```

```
        print i, fruits[i]
    }
}
```

Output:

```
1 apple
2 banana
3 mango
```

5.42 Formatted output with `printf`

`print` is simple, but `printf` provides formatting control.

```
awk '{ printf "%-12s %10d\n", $2, $4 }' employees.txt
```

Possible output:

```
Alice          75000
Bob            62000
Carol          82000
```

Format specifiers:

Specifier	Meaning
<code>%s</code>	String
<code>%d</code>	Integer
<code>%f</code>	Floating-point number
<code>%.2f</code>	Floating-point number with two decimal places
<code>%10s</code>	Right-aligned string in width 10
<code>%-10s</code>	Left-aligned string in width 10
<code>%06d</code>	Integer padded with zeros

Example:

```
awk '{ printf "%-10s ₹%.2f\n", $2, $4 }' employees.txt
```

5.43 Pass Bash variables to `awk`

Use `-v` :

```
minimum_salary=70000

awk -v minimum="$minimum_salary" '
$4 >= minimum {
    print $2, $4
}
' employees.txt
```

This is safer and clearer than injecting shell variables directly into the program.

Avoid:

```
awk '$4 >= "$minimum_salary"' { print $2 }' employees.txt
```

Although this can work, quoting becomes difficult and unsafe with complex values.

5.44 Multiple variables

```
department="Engineering"
minimum_salary=75000

awk
    -v department="$department"
    -v minimum="$minimum_salary" '
$3 == department && $4 >= minimum {
    print $2, $4
}
' employees.txt
```

5.45 Built-in `ARGV` and `ARGC`

`ARGC` contains the number of command-line arguments.

`ARGV` contains the arguments.

```
awk '
BEGIN {
    for (i = 0; i < ARGC; i++) {
        print "ARGV[" i "]" "=", ARGV[i]
    }
}
```



```
}  
}  
' file1.txt file2.txt
```

5.46 FILENAME

```
awk '{ print FILENAME, $0 }' file1.txt file2.txt
```

`FILENAME` contains the name of the current input file.

5.47 next

`next` stops processing the current record and reads the next one.

Skip the header:

```
awk 'NR == 1 { next } { print $2, $4 }' employees.txt
```

Another common format:

```
awk '  
NR == 1 {  
    next  
}  
{  
    print $2, $4  
}  
' employees.txt
```

Skip blank lines:

```
awk 'NF == 0 { next } { print }' file.txt
```

5.48 nextfile

`nextfile` stops processing the current file and moves to the next input file.

```
awk '  
/STOP/ {
```

```
    nextfile
}
{
    print FILENAME, $0
}
' file1.txt file2.txt
```

`nextfile` is widely supported in modern `awk` implementations but was not in the earliest POSIX specifications.

5.49 `exit`

Stop processing:

```
awk '/ERROR/ { print; exit }' application.log
```

This prints the first matching line and exits.

Use a custom exit status:

```
awk '
/ERROR/ {
    exit 1
}
END {
    if (NR > 0) {
        print "Processing finished"
    }
}
' application.log
```

Be careful: the `END` block usually still runs after `exit`.

5.50 `getline`

`getline` reads additional input.

Read from a file:

```
awk '
BEGIN {
    while ((getline line < "names.txt") > 0) {
        print line
    }
}
```

```
    close("names.txt")
}
```

Run a command and read its output:

```
awk '
BEGIN {
    command = "date"

    command | getline current_date

    close(command)

    print current_date
}
```

`getline` is powerful but can make programs harder to understand. Use normal input processing when possible.

5.51 User-defined functions

```
awk '
function calculate_bonus(salary, percentage) {
    return salary * percentage / 100
}

{
    bonus = calculate_bonus($4, 10)
    print $2, $4, bonus
}
' employees.txt
```

Function syntax:

```
function function_name(parameter1, parameter2) {
    statements
    return value
}
```

5.52 Mathematical functions

Common built-in functions:

Function	Meaning
<code>sqrt(x)</code>	Square root
<code>int(x)</code>	Integer portion
<code>log(x)</code>	Natural logarithm
<code>exp(x)</code>	Exponential
<code>sin(x)</code>	Sine
<code>cos(x)</code>	Cosine
<code>atan2(y, x)</code>	Arctangent
<code>rand()</code>	Random number
<code>srand()</code>	Seed random generator

Example:

```
awk 'BEGIN { print sqrt(81) }'
```

Output:

```
9
```

Generate random numbers:

```
awk '
BEGIN {
    srand()

    for (i = 1; i <= 5; i++) {
        print int(rand() * 100) + 1
    }
}
```

5.53 Dates in GNU `awk`

GNU `awk`, called `gawk`, provides date functions such as:

- `sysftime()`
- `strftime()`
- `mktime()`

Example:

```
gawk 'BEGIN { print strftime("%Y-%m-%d %H:%M:%S", sysftime()) }'
```

These are GNU extensions and may not exist in basic POSIX `awk`.

5.54 Process `/etc/passwd`

The `/etc/passwd` file uses `:` as the separator.

Print usernames:

```
awk -F: '{ print $1 }' /etc/passwd
```

Print username and shell:

```
awk -F: '{ print $1, $7 }' /etc/passwd
```

Print users whose shell is Bash:

```
awk -F: '$7 == "/bin/bash" { print $1 }' /etc/passwd
```

5.55 Process command output

Display filesystem usage greater than 80%:

```
df -P | awk '
NR > 1 {
    usage = $5
    gsub(/%/,"", usage)

    if (usage > 80) {
        print $1, usage "%"
```

```
}  
}  
,
```

Explanation:

1. `df -P` prints filesystem usage in a predictable format.
2. `NR > 1` skips the header.
3. `$5` contains usage such as `85%`.
4. `gsub(/%/ , "", usage)` removes the percent sign.
5. The condition checks whether usage is greater than 80.

5.56 Process logs

Suppose an access log starts with an IP address.

Count requests by IP:

```
awk '{ count[$1]++ } END {  
    for (ip in count) {  
        print ip, count[ip]  
    }  
}' access.log
```

Sort the result numerically:

```
awk '{ count[$1]++ } END {  
    for (ip in count) {  
        print count[ip], ip  
    }  
}' access.log | sort -nr
```

Why print the count first?

Because `sort -nr` sorts numerically based on the beginning of each line.

5.57 Remove duplicate lines with `awk`

```
awk '!seen[$0]++' file.txt
```

This compact command is important.

Explanation:

For a new line:

```
seen[$0]
```

Initially evaluates to zero.

The expression:

```
seen[$0]++
```

returns the old value, then increments it.

The `!` negates the old value.

For the first occurrence:

```
seen[line] = 0  
!0 = true
```

So the line is printed.

For later occurrences:

```
seen[line] = 1 or more  
!1 = false
```

So the line is not printed.

Because no action is specified, a true pattern performs the default action:

```
{ print $0 }
```

5.58 Join two files

Suppose `names.txt` contains:

```
101 Alice  
102 Bob  
103 Carol
```

And `salaries.txt` contains:

```
101 75000
102 62000
103 82000
```

Join them by ID:

```
awk '
NR == FNR {
    name[$1] = $2
    next
}
{
    print $1, name[$1], $2
}
' names.txt salaries.txt
```

Output:

```
101 Alice 75000
102 Bob 62000
103 Carol 82000
```

Explanation:

```
NR == FNR
```

This is true while reading the first file.

For the first file:

```
name[$1] = $2
```

stores each name by ID.

```
next
```

skips the remaining actions for the first file.

When the second file begins, `NR` continues increasing but `FNR` resets, so `NR == FNR` becomes false.

The second block then prints the joined values.

5.59 CSV warning

This command:

```
awk -F, '{ print $2 }' file.csv
```

works only for simple CSV data.

It can fail with proper quoted CSV such as:

```
101,"Mukherjee, Arko",Engineering,75000
```

The comma inside the quoted name is not a real field separator, but normal `awk -F,` will treat it as one.

For complex CSV, use:

- Python's `csv` module
- `csvkit`
- Miller
- A dedicated CSV parser
- GNU `awk` with careful FPAT logic

5.60 FPAT in GNU `awk`

GNU `awk` supports `FPAT`, which describes what a field looks like instead of what separates fields.

Example for basic CSV-like fields:

```
gawk '
BEGIN {
    FPAT = "([^,]*)|(\"[^\"]+\")"
}
{
    for (i = 1; i <= NF; i++) {
        print i, $i
    }
}
' file.csv
```

Even this does not implement every edge case of the CSV standard.

5.61 Run an external command

`awk` provides `system()`:

```
awk '
BEGIN {
    system("date")
}
```

Security warning: never build `system()` commands from untrusted input without strict validation.

5.62 Redirect output from `awk`

Write matching lines to another file:

```
awk '$4 > 70000 { print > "high_salary.txt" }' employees.txt
```

Append using `>>`:

```
awk '$4 > 70000 { print >> "high_salary.txt" }' employees.txt
```

Close files when dynamically generating many output files:

```
awk '{
    output_file = $3 ".txt"
    print $0 >> output_file
    close(output_file)
}' employees.txt
```

This creates files such as:

```
Engineering.txt
Sales.txt
Marketing.txt
```

5.63 Use an `awk` script file

Create `report.awk`:

```

BEGIN {
    print "Employee Salary Report"
    print "-----"
}

{
    total += $4
    count++

    printf "%-12s %10.2f\n", $2, $4
}

END {
    print "-----"

    if (count > 0) {
        printf "Average: %.2f\n", total / count
    }
}

```

Run:

```
awk -f report.awk employees.txt
```

You can also make the script executable.

Add a shebang:

```

#!/usr/bin/awk -f

BEGIN {
    print "Employee Salary Report"
}

{
    print $2, $4
}

```

Then:

```

chmod +x report.awk
./report.awk employees.txt

```

Shebang behavior can vary when additional interpreter arguments are required, so `awk -f script.awk file` is often the most portable form.

6. `find` Command

6.1 What is `find`?

`find` searches files and directories recursively.

General syntax:

```
find starting_path expression
```

Example:

```
find . -name "file.txt"
```

Here:

- `.` means start from the current directory.
- `-name "file.txt"` matches objects named `file.txt`.

6.2 List everything recursively

```
find .
```

This prints all files and directories under the current directory.

6.3 Find by exact name

```
find . -name "report.txt"
```

The search is case-sensitive.

6.4 Case-insensitive name search

```
find . -iname "report.txt"
```

This can match:

```
report.txt
Report.txt
REPORT.TXT
```

6.5 Find by wildcard pattern

Find all text files:

```
find . -name "*.txt"
```

The quotes are important.

Correct:

```
find . -name "*.txt"
```

Potentially incorrect:

```
find . -name *.txt
```

Without quotes, the shell may expand `*.txt` before `find` receives it.

6.6 Find only files

```
find . -type f
```

Common file types:

Type	Meaning
<code>f</code>	Regular file
<code>d</code>	Directory
<code>l</code>	Symbolic link
<code>b</code>	Block device
<code>c</code>	Character device
<code>p</code>	Named pipe
<code>s</code>	Socket

Find only directories:

```
find . -type d
```

Find symbolic links:

```
find . -type l
```

6.7 Combine name and type

```
find . -type f -name "*.sh"
```

This finds regular files ending in `.sh`.

Expressions next to each other normally have implicit AND behavior.

The command means:

```
type is regular file AND name ends in .sh
```

6.8 Logical AND

Implicit AND:

```
find . -type f -name "*.log"
```

Explicit AND:

```
find . -type f -a -name "*.log"
```

The explicit `-a` is usually unnecessary.

6.9 Logical OR

```
find . -type f \( -name "*.jpg" -o -name "*.png" \)
```

Explanation:

- `'and'` group expressions.
- `-o` means OR.
- The parentheses are escaped so the shell does not interpret them.

Find Python or shell scripts:

```
find . -type f \( -name "*.py" -o -name "*.sh" \)
```

6.10 Logical NOT

```
find . -type f ! -name "*.log"
```

This finds regular files whose names do not end in `.log`.

Alternative:

```
find . -type f -not -name "*.log"
```

6.11 Search by size

Find files larger than 100 megabytes:

```
find . -type f -size +100M
```

Find files smaller than 10 kilobytes:

```
find . -type f -size -10k
```

Find files of approximately exactly 1 megabyte in `find` units:

```
find . -type f -size 1M
```

Common units:

Unit	Meaning
<code>c</code>	Bytes

Unit	Meaning
k	KiB-sized blocks
M	MiB-sized blocks
G	GiB-sized blocks

Prefix meanings:

Prefix	Meaning
+	Greater than
-	Less than
No prefix	Exactly that number of units according to <code>find</code> rounding rules

Find files larger than 1 GB:

```
find /var -type f -size +1G
```

6.12 Search by modification time

Find files modified within the last 7 days:

```
find . -type f -mtime -7
```

Find files modified more than 30 days ago:

```
find . -type f -mtime +30
```

Find files modified approximately 7 whole 24-hour periods ago:

```
find . -type f -mtime 7
```

`-mtime` works in 24-hour units and uses rounding rules that can surprise beginners.

6.13 Search by minutes

Find files modified in the last 30 minutes:


```
find . -type f -mmin -30
```

Find files modified more than 120 minutes ago:

```
find . -type f -mmin +120
```

6.14 Access and metadata-change times

Option	Meaning
<code>-mtime</code>	File content modification time
<code>-atime</code>	Last access time
<code>-ctime</code>	Last metadata/status change time
<code>-mmin</code>	Modification age in minutes
<code>-amin</code>	Access age in minutes
<code>-cmin</code>	Metadata-change age in minutes

Important: `ctime` is not creation time.

It represents changes to metadata such as:

- Permissions
- Ownership
- Link count
- File content updates

6.15 Search relative to another file

Find files newer than `reference.txt`:

```
find . -type f -newer reference.txt
```

This is useful in backup and build scripts.

6.16 Search by permission

Find files with exactly permission mode `644`:

```
find . -type f -perm 0644
```

Find files where all executable bits in mode `111` are set:

```
find . -type f -perm -0111
```

Find files where any executable bit is set:

```
find . -type f -perm /0111
```

Find world-writable files:

```
find . -type f -perm -0002
```

Be cautious when changing permissions automatically.

6.17 Search by owner or group

Find files owned by user `alice`:

```
find /home -user alice
```

Find files owned by group `developers`:

```
find /srv -group developers
```

Find files with no valid user:

```
find / -nouser 2>/dev/null
```

Find files with no valid group:

```
find / -nogroup 2>/dev/null
```

6.18 Search empty files and directories

```
find . -empty
```

Only empty regular files:

```
find . -type f -empty
```

Only empty directories:

```
find . -type d -empty
```

6.19 Search symbolic links

```
find . -type l
```

Find broken symbolic links:

```
find . -xtype l
```

Support and precise behavior can vary between implementations.

6.20 Control search depth

Search only the current directory:

```
find . -maxdepth 1
```

Search at least two levels below the starting directory:

```
find . -mindepth 2
```

Find files exactly two levels deep:

```
find . -mindepth 2 -maxdepth 2 -type f
```

`-maxdepth` and `-mindepth` are commonly available in GNU `find` and several modern implementations but are not part of the oldest portable syntax.

6.21 Execute a command for each result

```
find . -type f -name "*.txt" -exec wc -l {} \;
```

Explanation:

- `-exec` starts an external command.
- `wc -l` counts lines.
- `{}` is replaced with the current pathname.
- `\;` ends the `-exec` expression.

This executes `wc -l` separately for each file.

6.22 Execute commands in batches

```
find . -type f -name "*.txt" -exec wc -l {} +
```

The `+` form passes multiple files to one command invocation.

This is often faster than `\;`.

Comparison:

```
-exec command {} \;
```

Runs one command per result.

```
-exec command {} +
```

Runs the command with batches of results.

6.23 Ask before executing

```
find . -type f -name "*.tmp" -ok rm {} \;
```

`-ok` asks for confirmation before running the command.

This can be useful while testing.

6.24 Delete matching files

```
find . -type f -name "*.tmp" -delete
```

Always preview before deleting:

```
find . -type f -name "*.tmp" -print
```

After confirming the output:

```
find . -type f -name "*.tmp" -delete
```

The position of `-delete` matters because `find` expressions are evaluated in order.

6.25 Delete old log files

Preview files older than 30 days:

```
find /var/log/myapp -type f -name "*.log" -mtime +30 -print
```

Delete after verification:

```
find /var/log/myapp -type f -name "*.log" -mtime +30 -delete
```

6.26 Remove empty directories

Preview:

```
find . -depth -type d -empty -print
```

Delete:

```
find . -depth -type d -empty -delete
```

`-depth` processes directory contents before the directory itself.

This is helpful when deleting directories.

6.27 Use `find` with `grep`

```
find . -type f -name "*.log" -exec grep -n "ERROR" {} +
```

This finds `.log` files and searches them for `ERROR`.

GNU `grep` can often do this more simply:

```
grep -rn --include='*.log' "ERROR" .
```

However, `find` gives more control over:

- Size
- Time
- Permissions
- Owner
- Directory pruning
- Complex file conditions

6.28 Use `find` with `sed`

Replace text inside all `.txt` files:

```
find . -type f -name "*.txt" -exec sed -i.bak 's/old/new/g' {} +
```

This creates backup files ending in `.bak`.

Preview first:

```
find . -type f -name "*.txt" -exec grep -Hn "old" {} +
```

6.29 Use `find` with `awk`

Count lines in all log files:

```
find . -type f -name "*.log" -exec awk '
{
    total++
}
END {
    print "Total lines:", total
}
```

```
}  
' {} +
```

Because all files are passed into one `awk` invocation in batches, `total` counts across those files for that invocation.

6.30 Use `-print0`

Normal pathnames can contain:

- Spaces
- Tabs
- Quotes
- Newlines
- Wildcards

To process arbitrary filenames safely, use null-byte separation.

```
find . -type f -print0
```

Combine it with `xargs -0`:

```
find . -type f -name "*.txt" -print0 |  
xargs -0 grep -n "ERROR"
```

This safely handles spaces and most unusual filename characters.

6.31 `find -exec` versus `xargs`

Safe and straightforward:

```
find . -type f -name "*.txt" -exec grep -n "ERROR" {} +
```

Using null-delimited `xargs`:

```
find . -type f -name "*.txt" -print0 |  
xargs -0 grep -n "ERROR"
```

Both can be efficient.

`-exec ... {} +` is often easier to write safely.

6.32 Avoid running `xargs` with no input

On GNU systems:

```
find . -type f -name "*.txt" -print0 |  
  xargs -0 -r grep -n "ERROR"
```

The `-r` option prevents `grep` from running when there are no files.

`xargs -r` is not portable to every Unix system.

6.33 Prune directories

Exclude `.git`:

```
find .  
  -path './.git' -prune  
  -o -type f -name "*.sh" -print
```

Explanation:

```
-path './.git' -prune
```

If the path is `.git`, do not descend into it.

```
-o
```

Otherwise evaluate the next expression.

```
-type f -name "*.sh" -print
```

Print matching shell scripts.

Exclude multiple directories:

```
find .  
  \( -path './.git' -o -path './node_modules' -o -path './venv' \)  
  -prune  
  -o  
  -type f -name "*.py" -print
```

6.34 Search by path

```
find . -path "**/tests/*.py"
```

This matches Python files under directories named `tests`.

Case-insensitive path search on supporting systems:

```
find . -ipath "**/TESTS/*.py"
```

6.35 Search with regular expressions

GNU `find`:

```
find . -regextype posix-extended  
-regex '.*\.(jpg|jpeg|png)$'
```

Important: `-regex` usually matches the complete path, not only the filename.

That is why the pattern begins with:

```
.*
```

For simple extension matching, `-name` is usually easier:

```
find . -type f \( -name "*.jpg" -o -name "*.jpeg" -o -name "*.png" \)
```

6.36 Custom output with `-printf`

GNU `find` supports `-printf`.

```
find . -type f -printf '%p %s bytes\n'
```

Common format codes:

Code	Meaning
<code>%p</code>	Full pathname

Code	Meaning
<code>%f</code>	Filename
<code>%s</code>	Size in bytes
<code>%u</code>	Owner username
<code>%g</code>	Group name
<code>%m</code>	Permission mode
<code>%TY-%Tm-%Td</code>	Modification date
<code>%TH:%TM:%TS</code>	Modification time

Example:

```
find . -type f -printf '%TY-%Tm-%Td %TH:%TM %10s %p\n'
```

`-printf` is not POSIX and may not be available on BSD/macOS `find`.

Portable alternative:

```
find . -type f -exec ls -ld {} \;
```

Though `ls` output is less reliable for machine parsing.

6.37 Find the largest files

GNU-style approach:

```
find . -type f -printf '%s %p\n' |
  sort -nr |
  head -10
```

Explanation:

1. `find` prints file size and path.
2. `sort -nr` sorts numerically in descending order.
3. `head -10` displays the largest ten.

This simple form can be confused by filenames containing newlines.

6.38 Find recently modified files

Files modified in the last hour:

```
find . -type f -mmin -60
```

Files modified today can be approximated using a reference timestamp:

```
touch -d 'today 00:00' /tmp/today_start
```

```
find . -type f -newer /tmp/today_start
```

`touch -d` is a GNU feature.

6.39 Find executable files

```
find . -type f -executable
```

This is a GNU-style predicate.

A permission-based alternative:

```
find . -type f -perm /111
```

6.40 Find files with multiple conditions

Find log files larger than 100 MB and older than 7 days:

```
find /var/log
  -type f
  -name "*.log"
  -size +100M
  -mtime +7
```

All conditions must be true.

7. Combining `grep`, `sed`, `awk`, and `find`

7.1 Find files and search their contents

```
find . -type f -name "*.conf" -exec grep -Hn "password" {} +
```

Explanation:

1. `find` locates configuration files.
2. `-exec` passes those files to `grep`.
3. `-H` prints filenames.
4. `-n` prints matching line numbers.

7.2 Search, extract, and summarize logs

Suppose log lines look like:

```
2026-07-15 INFO User login successful
2026-07-15 ERROR Database unavailable
2026-07-15 WARNING Memory usage high
2026-07-15 ERROR Login failed
```

Count messages by level:

```
grep -E 'INFO|WARNING|ERROR' application.log |
awk '{ count[$2]++ }
     END {
         for (level in count) {
             print level, count[level]
         }
     }'
```

Explanation:

1. `grep` selects relevant log levels.
2. `awk` uses the second field as the category.
3. `count[$2]++` counts each category.
4. The `END` block prints the totals.

7.3 Search files, replace text, and preserve backups

Preview:

```
find . -type f -name "*.sh" -exec grep -Hn "old_command" {} +
```

Replace:

```
find . -type f -name "*.sh"  
-exec sed -i.bak 's/old_command/new_command/g' {} +
```

The backup files allow recovery.

7.4 Extract errors and remove prefixes

Input:

```
2026-07-15 12:00:01 ERROR Database unavailable  
2026-07-15 12:00:02 INFO Login successful  
2026-07-15 12:00:03 ERROR Login failed
```

Command:

```
grep 'ERROR' application.log |  
sed -E 's/^[0-9-]+ [0-9:]+ ERROR[[:space:]]+//'
```

Output:

```
Database unavailable  
Login failed
```

7.5 Calculate totals from matching records

```
grep "Engineering" employees.txt |  
awk '{ total += $4 }  
END { print "Engineering salary total:", total }'
```

`grep` filters the records, and `awk` performs the calculation.

However, `awk` can do both by itself:

```
awk '$3 == "Engineering" {  
    total += $4  
}
```

```
END {  
    print "Engineering salary total:", total  
}' employees.txt
```

Use fewer commands when one tool can clearly perform the complete task.

7.6 Find logs and generate an error report

```
find /var/log/myapp  
-type f  
-name "*.log"  
-mtime -7  
-exec grep -Hn "ERROR" {} + |  
awk -F: '  
{  
    file = $1  
    error_count[file]++  
}  
END {  
    for (file in error_count) {  
        print file, error_count[file]  
    }  
}'
```

This:

1. Finds log files modified within seven days.
2. Searches them for `ERROR`.
3. Counts matching lines by filename.

Caution: splitting `grep` output with `-F:` can fail if filenames contain colons.

A safer architecture is to process files directly with `awk` or use null-delimited output where supported.

7.7 Extract unique IP addresses

```
grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log |  
sort -u
```

Using `awk` when the IP is the first field:

```
awk '!seen[$1]++ { print $1 }' access.log
```

7.8 Replace configuration values

Suppose:

```
PORT=8080
HOST=localhost
DEBUG=true
```

Replace the port:

```
sed -E 's/^PORT=.* /PORT=9090/' application.conf
```

Modify the file with a backup:

```
sed -i.bak -E 's/^PORT=.* /PORT=9090/' application.conf
```

8. Practical Bash Scripts

8.1 Search text in selected file types

```
#!/usr/bin/env bash

set -euo pipefail

search_text=${1:-}
search_directory=${2:-.}

if [[ -z "$search_text" ]]; then
    echo "Usage: $0 SEARCH_TEXT [DIRECTORY]" >&2
    exit 1
fi

grep
    -Rni
    --include='*.sh'
    --include='*.py'
    --include='*.txt'
    --exclude-dir='.git'
    --exclude-dir='node_modules'
    -- "$search_text" "$search_directory"
```

Explanation

```
#!/usr/bin/env bash
```

Runs the script using Bash located through the current environment.

```
set -euo pipefail
```

Enables stricter error handling:

- `-e` : stop when an unhandled command fails.
- `-u` : report unset variables.
- `pipefail` : a pipeline fails when one of its commands fails.

```
search_text=${1:-}
```

Stores the first command-line argument.

`${1:-}` returns an empty string if `$1` is unset.

```
search_directory=${2:-.}
```

Uses the second argument as the directory or defaults to `.`.

```
if [[ -z "$search_text" ]]; then
```

Checks whether the search text is empty.

```
-- "$search_text"
```

The `--` marks the end of options. This protects patterns beginning with `-`.

The quotes preserve spaces and wildcard characters in the user's input.

8.2 Report large files

```
#!/usr/bin/env bash

set -euo pipefail

search_directory=${1:-.}
```



```

minimum_size=${2:-100M}

printf 'Files larger than %s under %s\n\n'
    "$minimum_size" "$search_directory"

find "$search_directory"
    -type f
    -size "+$minimum_size"
    -printf '%12s bytes  %p\n' |
sort -nr

```

Important note

The expression:

```
-size "+$minimum_size"
```

produces values such as:

```
+100M
```

The script depends on GNU `find` because it uses `-printf`.

8.3 Clean old log files safely

```

#!/usr/bin/env bash

set -euo pipefail

log_directory=${1:-/var/log/myapp}
retention_days=${2:-30}

if [[ ! -d "$log_directory" ]]; then
    echo "Directory does not exist: $log_directory" >&2
    exit 1
fi

echo "The following files are older than $retention_days days:"

find "$log_directory"
    -type f
    -name "*.log"
    -mtime "+$retention_days"
    -print

read -r -p "Delete these files? [y/N] " answer

```

```

if [[ "$answer" == "y" || "$answer" == "Y" ]]; then
    find "$log_directory"
        -type f
        -name "*.log"
        -mtime "+$retention_days"
        -delete

    echo "Old log files deleted."
else
    echo "No files were deleted."
fi

```

This script previews the files before deletion.

8.4 Employee salary report

```

#!/usr/bin/env bash

set -euo pipefail

input_file=${1:-employees.txt}

if [[ ! -f "$input_file" ]]; then
    echo "File not found: $input_file" >&2
    exit 1
fi

awk '
BEGIN {
    printf "%-8s %-15s %-15s %12s\n",
        "ID", "Name", "Department", "Salary"

    printf "%-8s %-15s %-15s %12s\n",
        "-----",
        "-----",
        "-----",
        "-----"
}

{
    id = $1
    name = $2
    department = $3
    salary = $4

    total += salary
    department_total[department] += salary
}
'

```

```

        department_count[department]++

        if (NR == 1 || salary > maximum_salary) {
            maximum_salary = salary
            maximum_employee = name
        }

        printf "%-8s %-15s %-15s %12.2f\n",
            id, name, department, salary
    }

END {
    print ""
    printf "Number of employees: %d\n", NR

    if (NR > 0) {
        printf "Total salary: %.2f\n", total
        printf "Average salary: %.2f\n", total / NR
        printf "Highest salary: %s, %.2f\n",
            maximum_employee, maximum_salary
    }

    print ""
    print "Department summary:"

    for (department in department_total) {
        average = department_total[department] /
            department_count[department]

        printf "%-15s Count: %d Total: %.2f Average: %.2f\n",
            department,
            department_count[department],
            department_total[department],
            average
    }
}
' "$input_file"

```

8.5 Configuration updater

```

#!/usr/bin/env bash

set -euo pipefail

configuration_file=${1:-application.conf}
new_port=${2:-9090}

if [[ ! -f "$configuration_file" ]]; then

```

```

        echo "Configuration file not found: $configuration_file" >&2
        exit 1
    fi

    if [[ ! "$new_port" =~ ^[0-9]+$ ]]; then
        echo "Port must contain only digits." >&2
        exit 1
    fi

    if (( new_port < 1 || new_port > 65535 )); then
        echo "Port must be between 1 and 65535." >&2
        exit 1
    fi

    sed -i.bak -E
        "s/^PORT=.*PORT=$new_port/"
        "$configuration_file"

    echo "Port updated to $new_port."
    echo "Backup created at ${configuration_file}.bak"

```

The Bash variable is inserted into the `sed` program using double quotes.

This is acceptable here because the port has been validated as digits only.

8.6 Log-analysis script

```

#!/usr/bin/env bash

set -euo pipefail

log_file=${1:-application.log}

if [[ ! -f "$log_file" ]]; then
    echo "Log file not found: $log_file" >&2
    exit 1
fi

awk '
{
    level = $2
    count[level]++
}

$2 == "ERROR" {
    error_messages[$0]++
}
'

```

```

END {
    print "Log Level Summary"
    print "-----"

    for (level in count) {
        printf "%-10s %d\n", level, count[level]
    }

    print ""
    print "Repeated Error Messages"
    print "-----"

    for (message in error_messages) {
        printf "%d %s\n", error_messages[message], message
    }
}
' "$log_file"

```

8.7 Search recent large log files for errors

```

#!/usr/bin/env bash

set -euo pipefail

search_directory=${1:-.}

find "$search_directory"
    -type f
    -name "*.log"
    -mtime -7
    -size +1M
    -exec grep -Hn "ERROR" {} +

```

This searches:

- Only regular files
- Only `.log` files
- Only files modified within seven days
- Only files larger than one megabyte
- Only lines containing `ERROR`

9. Common Mistakes and Safety Guidelines

9.1 Forgetting to quote wildcard patterns

Incorrect:

```
find . -name *.txt
```

Correct:

```
find . -name "*.txt"
```

The quotes prevent Bash from expanding the wildcard too early.

9.2 Unquoted variables

Dangerous:

```
find $directory -name "*.txt"
```

Safer:

```
find "$directory" -name "*.txt"
```

If the directory contains spaces, the unquoted version breaks it into multiple arguments.

9.3 Using `for file in $(find ...)`

Avoid:

```
for file in $(find . -type f); do
    echo "$file"
done
```

This breaks filenames containing:

- Spaces
- Tabs
- Newlines
- Wildcard characters

Use a null-delimited loop:

```
while IFS= read -r -d '' file; do
    printf '%s\n' "$file"
done < <(find . -type f -print0)
```

9.4 Using `cat` unnecessarily

Less direct:

```
cat file.txt | grep "ERROR"
```

Better:

```
grep "ERROR" file.txt
```

Less direct:

```
cat employees.txt | awk '{ print $2 }'
```

Better:

```
awk '{ print $2 }' employees.txt
```

Piping from `cat` is not always wrong, but it is usually unnecessary when the command accepts filenames directly.

9.5 Editing files without backups

Risky:

```
sed -i 's/old/new/g' file.txt
```

Safer during testing:

```
sed -i.bak 's/old/new/g' file.txt
```

Also preview first:

```
sed 's/old/new/g' file.txt
```

9.6 Deleting directly with `find`

Risky:

```
find . -name "*.tmp" -delete
```

Preview first:

```
find . -name "*.tmp" -print
```

Only after checking:

```
find . -name "*.tmp" -delete
```

9.7 Confusing `ctime` with creation time

This:

```
find . -ctime -1
```

does not necessarily find files created within one day.

It finds files whose metadata status changed within one day.

9.8 Parsing `ls` output

Avoid processing filenames with:

```
ls | grep ...
```

`ls` output is designed for humans and can be unreliable for machine parsing.

Use `find`, shell globs, or null-delimited data instead.

9.9 Using regex when fixed strings are intended

Suppose you want to search for:

```
file[1].txt
```

Using normal regex:

```
grep 'file[1].txt' file.txt
```

does not treat the brackets and dot literally.

Use fixed-string mode:

```
grep -F 'file[1].txt' file.txt
```

9.10 Forgetting that `grep` matches lines

```
grep -c "ERROR" file.txt
```

counts matching lines, not necessarily all occurrences.

Count occurrences:

```
grep -o "ERROR" file.txt | wc -l
```

9.11 Using `grep` when `awk` is clearer

Less efficient:

```
grep "Engineering" employees.txt |  
  awk '$4 > 70000 { print $2 }'
```

Clearer:

```
awk '$3 == "Engineering" && $4 > 70000 {  
    print $2  
}' employees.txt
```

9.12 Assuming every Unix system uses GNU tools

GNU and BSD/macOS versions differ.

Common differences include:

- `sed -i`
- `find -printf`
- `find -regextype`
- `grep -P`
- `date -d`
- Some regex flags
- Some `awk` extensions

For portable scripts:

- Prefer POSIX syntax.
- Test on the target operating systems.
- Avoid GNU-only features unless GNU tools are guaranteed.
- Document required dependencies.

9.13 Filenames beginning with `-`

Suppose a file is named:

```
-important.txt
```

Use:

```
grep "text" -- -important.txt
```

The `--` marks the end of options.

Another possibility:

```
grep "text" ./-important.txt
```

9.14 Command injection risks

Dangerous:

```
awk '{ system("rm " $1) }' files.txt
```

If input is untrusted, it could contain shell syntax or unexpected paths.

Prefer direct file-processing tools and strict validation.

10. Quick Reference Cheat Sheet

10.1 `grep` cheat sheet

```
# Search text
grep "pattern" file.txt

# Ignore case
grep -i "pattern" file.txt

# Show line numbers
grep -n "pattern" file.txt

# Invert match
grep -v "pattern" file.txt

# Count matching lines
grep -c "pattern" file.txt

# Match complete words
grep -w "word" file.txt

# Match complete lines
grep -x "complete line" file.txt

# Extended regex
grep -E 'error|warning' file.txt

# Fixed-string search
grep -F 'literal.string' file.txt

# Print only matched text
grep -oE '[0-9]+' file.txt

# Recursive search
grep -rn "pattern" directory/
```

```
# Include file type
grep -rn --include='*.py' "pattern" .

# Exclude directory
grep -rn --exclude-dir='.git' "pattern" .

# Show context
grep -C 2 "ERROR" file.txt

# Quiet existence check
grep -q "pattern" file.txt
```

10.2 `sed` cheat sheet

```
# Replace first match on each line
sed 's/old/new/' file.txt

# Replace every match
sed 's/old/new/g' file.txt

# Print specific line
sed -n '5p' file.txt

# Print range
sed -n '5,10p' file.txt

# Delete line
sed '5d' file.txt

# Delete matching lines
sed '/pattern/d' file.txt

# Print matching lines
sed -n '/pattern/p' file.txt

# Insert before line
sed '3i New line' file.txt

# Append after line
sed '3a New line' file.txt

# Replace complete line
sed '3c New content' file.txt

# Edit with backup
sed -i.bak 's/old/new/g' file.txt
```

```
# Remove blank lines
sed '/^[[:space:]]*$/d' file.txt

# Remove leading spaces
sed 's/^[[:space:]]*//' file.txt

# Remove trailing spaces
sed 's/[[:space:]]*$//' file.txt
```

10.3 `awk` cheat sheet

```
# Print complete line
awk '{ print $0 }' file.txt

# Print first field
awk '{ print $1 }' file.txt

# Print last field
awk '{ print $NF }' file.txt

# Use comma separator
awk -F, '{ print $2 }' file.csv

# Print matching records
awk '/ERROR/' file.txt

# Match a field
awk '$3 == "Engineering"' employees.txt

# Numeric comparison
awk '$4 > 70000 { print $2 }' employees.txt

# Sum values
awk '{ total += $4 } END { print total }' employees.txt

# Calculate average
awk '{ total += $4 } END {
    if (NR > 0) print total / NR
}' employees.txt

# Count by category
awk '{ count[$3]++ } END {
    for (key in count) print key, count[key]
}' employees.txt

# Pass a shell variable
awk -v limit="$limit" '$4 > limit' employees.txt
```

```
# Formatted output
awk '{ printf "%-15s %10.2f\n", $2, $4 }' employees.txt

# Remove duplicate lines
awk '!seen[$0]++' file.txt
```

10.4 `find` cheat sheet

```
# Find by name
find . -name "file.txt"

# Case-insensitive name
find . -iname "file.txt"

# Find regular files
find . -type f

# Find directories
find . -type d

# Find by extension
find . -type f -name "*.txt"

# Find larger than 100 MB
find . -type f -size +100M

# Modified within 7 days
find . -type f -mtime -7

# Modified within 30 minutes
find . -type f -mmin -30

# Empty files
find . -type f -empty

# Execute a command
find . -type f -name "*.txt" -exec wc -l {} +

# Delete after previewing
find . -type f -name "*.tmp" -delete

# Exclude .git
find . -path './.git' -prune -o -type f -print

# Null-delimited output
find . -type f -print0
```

```
# Search file contents
find . -type f -name "*.log" -exec grep -Hn "ERROR" {} +
```

Final Command Selection Guide

Use `grep` when the main question is:

Which lines contain this text or pattern?

Use `sed` when the main question is:

How can I replace, delete, insert, or transform text?

Use `awk` when the main question is:

How can I work with fields, conditions, calculations, categories, or reports?

Use `find` when the main question is:

Where are the files or directories matching these conditions?

Examples:

```
# Search text
grep "ERROR" application.log

# Replace text
sed 's/ERROR/CRITICAL/g' application.log

# Calculate from columns
awk '$2 == "ERROR" { count++ } END { print count }' application.log

# Locate log files
find /var/log -type f -name "*.log"
```

The commands can also be combined:

```
find /var/log/myapp
  -type f
  -name "*.log"
  -mtime -7
  -exec grep -Hn "ERROR" {} +
```

This command means:

1. Search inside `/var/log/myapp`.
2. Select regular files.
3. Select files ending in `.log`.
4. Select files modified within seven days.
5. Search those files for `ERROR`.
6. Print filenames and line numbers for every match.